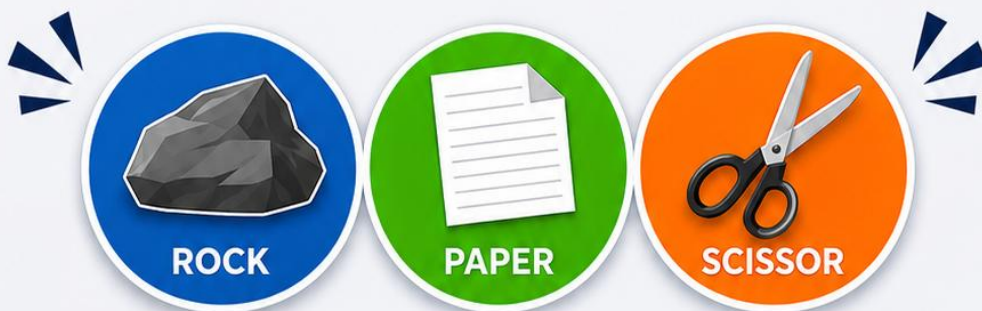


BESANT TECHNOLOGY

MINI PROJECT

ROCK PAPER SCISSOR



SUBMITTED TO :

Gowthami Shyam



SUBMITTED BY :

Gomathi Pandy.



DATE OF SUBMISSION :

28-May-2026



GITHUB LINK :

https://github.com/pgomathipandy/Rock_Paper_scissor



GitHub Link: https://github.com/pgomathipandy/Rock_Paper_scissor

Acknowledgement

I would like to express my sincere gratitude to Mrs. Gowthami Shyam for her valuable guidance, encouragement, and continuous support throughout the development of this mini project titled “Rock Paper Scissor Game using Python.”

I am thankful to [Besant Technology](#) for providing the opportunity, resources, and learning environment to enhance my programming and software engineering skills.

I also extend my heartfelt thanks to all faculty members, friends, and well-wishers who directly or indirectly helped me in completing this project successfully.

This project helped me gain practical knowledge in Python programming, logical problem-solving, exception handling, and software development concepts

Index

Contents

Introduction to Python.....	4
Features of Python.....	4
Applications of Python.....	5
Python in Software Engineering.....	5
Advantages of Python	6
Project Overview.....	7
Rules and Regulations of the Rock Paper Scissor Game	7
Concepts Used in the Source Code	9
1. Variables.....	9
2. Lists.....	9
3. Input Function.....	9
4. String Methods.....	9
5. Random Module	10
6. Conditional Statements.....	10
7. Logical Operators	11
8. Membership Operator	11
9. Exception Handling	12
10. Output Statements.....	13
11. Program Flow Control	13
12. Comments	14
Source code.....	15
Explanation of Source Code	16
Output.....	20
Conclusion.....	21
References.....	22

Introduction to Python

Python is a high-level, interpreted, and general-purpose programming language developed by Guido van Rossum and first released in 1991.

Python is widely known for its:

- Simple syntax
- Easy readability
- Beginner-friendly nature
- Powerful libraries and frameworks

It allows developers to write programs with fewer lines of code compared to many other programming languages.

Features of Python

1. Easy to Learn

Python uses simple English-like syntax, making it easy for beginners and professionals.

Example:

```
print("Hello World")
```

2. Interpreted Language

Python code is executed line by line using an interpreter, which simplifies debugging and testing.

3. Platform Independent

Python programs can run on:

- Windows
 - Linux
 - macOS
 - without major modifications.
-

4. Object-Oriented Language

Python supports object-oriented programming concepts such as:

- Classes
 - Objects
 - Inheritance
 - Polymorphism
-

5. Large Standard Library

Python provides built-in libraries for:

- Mathematics
- File handling
- Web development
- Data analysis
- Artificial Intelligence
- Machine Learning

Applications of Python

Python is used in many real-world applications:

Domain	Usage
Web Development	Websites and APIs
Data Science	Data analysis and visualization
Artificial Intelligence	Machine Learning and AI models
Automation	Task automation scripts
Game Development	Building games
Cyber Security	Security tools and testing
Software Development	Desktop and enterprise applications

Python in Software Engineering

In software engineering, Python is used for:

- Backend development
- Automation testing
- Scripting
- Data processing
- API development
- DevOps automation
- Cloud applications

Python helps developers build scalable and maintainable software systems efficiently.

Advantages of Python

- Benefits
- Simple and readable code
- Faster development process
- High productivity
- Strong community support
- Easy integration with other technologies
- Suitable for beginners and professionals

Project Overview

The **Rock Paper Scissor Game** is a simple command-line based mini project developed using Python. The project is designed to simulate the traditional Rock Paper Scissor game between a user and the computer.

In this application, the user selects one option among:

- Rock
- Paper
- Scissor

The computer automatically generates a random choice using Python's built-in random module. The program then compares both choices and determines the winner based on predefined game rules.

This project demonstrates fundamental software engineering and programming concepts such as:

- User interaction
- Conditional statements
- Random number generation
- Input validation
- Exception handling
- Program flow control

The primary objective of this project is to develop a beginner-friendly interactive application while improving logical thinking and coding skills. The project also provides practical exposure to Python programming and helps in understanding how real-world software applications process inputs, apply business logic, and generate outputs.

Additionally, exception handling techniques are implemented to make the application more stable and user-friendly by preventing crashes caused by invalid inputs.

This mini project serves as a foundational step toward developing more advanced software applications such as games, automation tools, and interactive systems.

Rules and Regulations of the Rock Paper Scissor Game

Game Rules

The Rock Paper Scissor game follows a simple set of predefined rules between the player and the computer.

Basic Rules

Rule 1: Rock beats Scissor

- Rock can break Scissor.
- Therefore, Rock wins.

Example:

Rock vs Scissor → Rock Wins

Rule 2: Paper beats Rock

- Paper can cover Rock.
- Therefore, Paper wins.

Example:

Paper vs Rock → Paper Wins

Rule 3: Scissor beats Paper

- Scissor can cut Paper.
- Therefore, Scissor wins.

Example:

Scissor vs Paper → Scissor Wins

Tie Condition

If both the user and the computer select the same option, the game results in a tie.

Example:

Rock vs Rock → Tie

Game Logic Table:

User Choice	Computer Choice	Result
Rock	Scissor	User Wins
Rock	Paper	Computer Wins
Paper	Rock	User Wins
Paper	Scissor	Computer Wins
Scissor	Paper	User Wins
Scissor	Rock	Computer Wins

Concepts Used in the Source Code

The Rock Paper Scissor Game project uses several important programming concepts of Python. These concepts help in understanding the fundamentals of software development and problem-solving.

1. Variables

Variables are used to store data values in memory.

Example :

```
user = "rock"
```

```
computer = "paper"
```

Purpose in Project

- Store user choice
- Store computer-generated choice
- Store temporary data during execution

2. Lists

A list is a collection of multiple values stored in a single variable.

Example:

```
choices = ["rock", "paper", "scissors"]
```

Purpose in Project:

- Store valid game options
- Allow random selection
- Simplify validation logic

3. Input Function

The input() function is used to receive data from the user.

Example:

```
user = input("Enter rock, paper, or scissors: ")
```

Purpose in Project :

- Accept user interaction
- Make the game interactive

4. String Methods

a) lower()

Converts text into lowercase.

Example

```
user.lower()
```

Purpose in Project :

- Ensures input consistency.

Example:

ROCK → rock

b) strip()

Removes extra spaces from the beginning and end of a string.

Example

user.strip()

Purpose in Project :

- Prevents errors caused by unwanted spaces.

Example:

" rock " → "rock"

5. Random Module

- The random module generates random values.
- Before using the random module, it must be imported.
 - `import random`
- This gives access to all functions available inside the module.
- `random.choice()` function selects one random element from a list, tuple, or string.

Example

computer = random.choice(choices)

Purpose in Project :

- Generate computer moves randomly
- Ensure fair gameplay

6. Conditional Statements

Conditional statements are used for decision making.

Statements Used :

- if
- elif
- else

Example

if user == computer:

Purpose in Project :

- Compare choices
- Determine winner
- Validate inputs

7. Logical Operators

Logical operators in Python are used to combine multiple conditions and return either:

- True
- False

They are mainly used in:

- Decision making
- Conditional statements
- Validation logic

Types of Logical Operators

Python provides three main logical operators:

Operator	Meaning
And	Returns True if both conditions are True
Or	Returns True if at least one condition is True
Not	Reverses the condition

Example

(user == "rock" and computer == "scissors")

Purpose

- Check multiple conditions simultaneously.

8. Membership Operator

The Membership Operator is used to check whether a value exists inside a sequence such as:

- List
- Tuple
- String
- Dictionary

It returns:

- True
- False

depending on whether the value is present or not.

Types of Membership Operators

Python provides two membership operators:

Operator	Meaning
in	Returns True if value exists
not in	Returns True if value does not exist

Purpose in Project :

The membership operator is used for:

- Input validation
- Checking whether the entered value is valid

Example :

```
print(user in choices)
```

9. Exception Handling

Exception Handling is a mechanism used to handle runtime errors and prevent the program from crashing unexpectedly.

It allows the program to:

- Detect errors
- Handle errors gracefully
- Continue execution smoothly

Exception handling improves:

- Software reliability
- Stability
- User experience

Exception Handling Keywords

Python uses the following keywords:

Keyword	Purpose
Try	Contains risky code
Except	Handles errors
finally	Executes always
Raise	Generates custom exceptions

Example

try:

```
user = input()
```

except:

```
print("Error")
```

Purpose in Project:

- Handle invalid inputs
- Improve software reliability
- Prevent runtime crashes

ValueError Exception

ValueError occurs when invalid values are provided.

Example

```
raise ValueError("Invalid choice entered")
```

Purpose:

- Generate custom error messages for wrong user inputs.

10. Output Statements

The print() function displays output on the screen.

Example

```
print("You Win")
```

Purpose:

Display:

- ❖ User choices
- ❖ Computer choices
- ❖ Game results
- ❖ Error messages

11. Program Flow Control

Flow control manages the sequence of program execution.

Purpose in Project

The program follows this flow:

- Get input
- Generate random choice
- Compare choices
- Display result

12. Comments

Comments in Python are non-executable statements used to explain the code. They improve the readability and understanding of programs.

Comments are ignored by the Python interpreter during execution.

They are mainly used for:

- Explaining logic
- Improving code readability
- Documentation
- Debugging
- Maintenance

Types of Comments in Python

Python supports:

1. Single-line comments
 - Single-line comments begin with the # symbol.
2. Multi-line comments
 - Multi-line comments are used for longer explanations.
 - Python uses triple quotes:
 - ''' '''
 - """ """

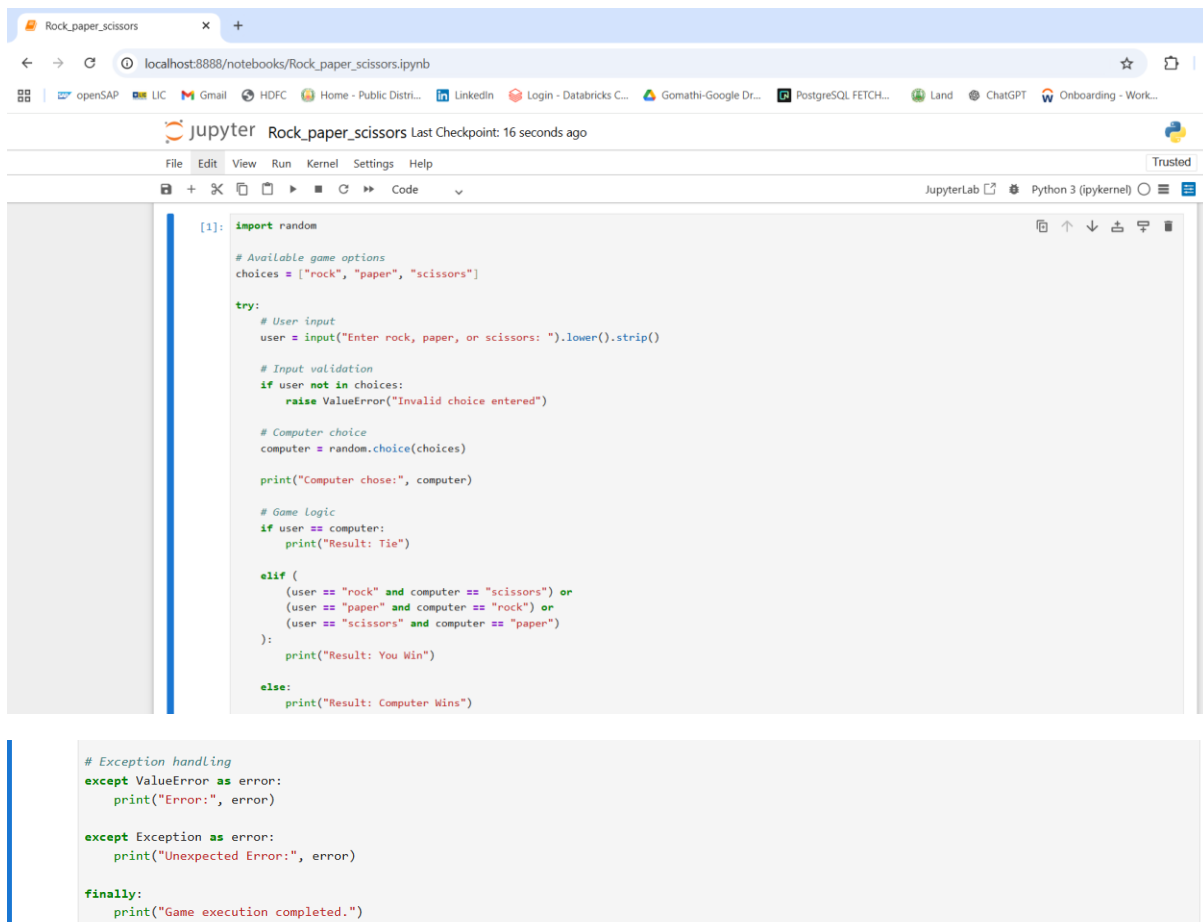
Example :

```
# User input
user = input("Enter your name: ")
```

Purpose

- Explain logic
- Improve maintainability
- Help debugging

Source code



```
[1]: import random

# Available game options
choices = ["rock", "paper", "scissors"]

try:
    # User input
    user = input("Enter rock, paper, or scissors: ").lower().strip()

    # Input validation
    if user not in choices:
        raise ValueError("Invalid choice entered")

    # Computer choice
    computer = random.choice(choices)

    print("Computer chose:", computer)

    # Game Logic
    if user == computer:
        print("Result: Tie")

    elif (
        (user == "rock" and computer == "scissors") or
        (user == "paper" and computer == "rock") or
        (user == "scissors" and computer == "paper")
    ):
        print("Result: You Win")

    else:
        print("Result: Computer Wins")

# Exception handling
except ValueError as error:
    print("Error:", error)

except Exception as error:
    print("Unexpected Error:", error)

finally:
    print("Game execution completed.")
```

Explanation of Source Code

This program is developed using Python to implement the **Rock Paper Scissor Game** with proper exception handling and input validation.

The program allows the user to play against the computer, where the computer generates a random choice and the program decides the winner based on game rules.

1. Importing Random Module

```
import random
```

Explanation

- Imports Python's built-in random module.
- Used to generate random computer choices.

Purpose

The computer should make unpredictable selections to ensure fair gameplay.

2. Creating Game Options List

```
choices = ["rock", "paper", "scissors"]
```

Explanation

A list named choices is created containing all valid game options.

Purpose

- Store valid inputs
- Help in random selection
- Simplify input validation

3. Try Block

```
try:
```

Explanation

The try block contains code that may generate errors during execution.

Purpose

- Prevent sudden program crashes
- Handle runtime exceptions safely

4. User Input

```
user = input("Enter rock, paper, or scissors: ").lower().strip()
```

Explanation

This line:

1. Accepts input from the user
2. Converts input into lowercase using: lower()
3. Removes extra spaces using : strip()

Example**User Input**

ROCK

After Processing

rock

5. Input Validation

if user not in choices:

Explanation

Checks whether the entered value exists in the valid options list.

Membership Operator Used

not in

Purpose

Detect invalid inputs.

6. Raising Custom Exception

raise ValueError("Invalid choice entered")

Explanation

Manually generates a ValueError exception when invalid input is entered.

Example Invalid Input

apple

Output

Error: Invalid choice entered

7. Computer Random Choice

computer = random.choice(choices)

Explanation

The computer randomly selects one value from:

- rock
- paper
- scissors

Function Used

random.choice()

8. Display Computer Choice

print("Computer chose:", computer)

Explanation

Displays the computer's selected option on the screen.

9. Tie Condition

if user == computer:

Explanation

Checks whether:

- User choice
- Computer choice

are the same.

Result

Result: Tie

10. Winning Conditions

elif (

(user == "rock" and computer == "scissors") or

(user == "paper" and computer == "rock") or

(user == "scissors" and computer == "paper")

)

Explanation

Checks all possible user-winning combinations.

Logical Operators Used

AND Operator

and

Used to combine two conditions.

Example:

user == "rock" and computer == "scissors"

OR Operator

or

Used to check multiple winning possibilities.

11. User Wins

print("Result: You Win")

Explanation

Executes when any winning condition becomes true.

12. Computer Wins

else:

print("Result: Computer Wins")

Explanation

Executes when:

- It is not a tie
 - User does not satisfy winning conditions
-

Therefore, the computer wins.

13. Exception Handling Block

except ValueError as error:

Explanation

Handles invalid input errors safely.

Example

apple

Output

Error: Invalid choice entered

14. Generic Exception Block

except Exception as error:

Explanation

Handles unexpected runtime errors that are not specifically handled earlier.

Purpose

Improves software stability and reliability.

15. Finally Block

finally:

print("Game execution completed.")

Explanation

The finally block always executes:

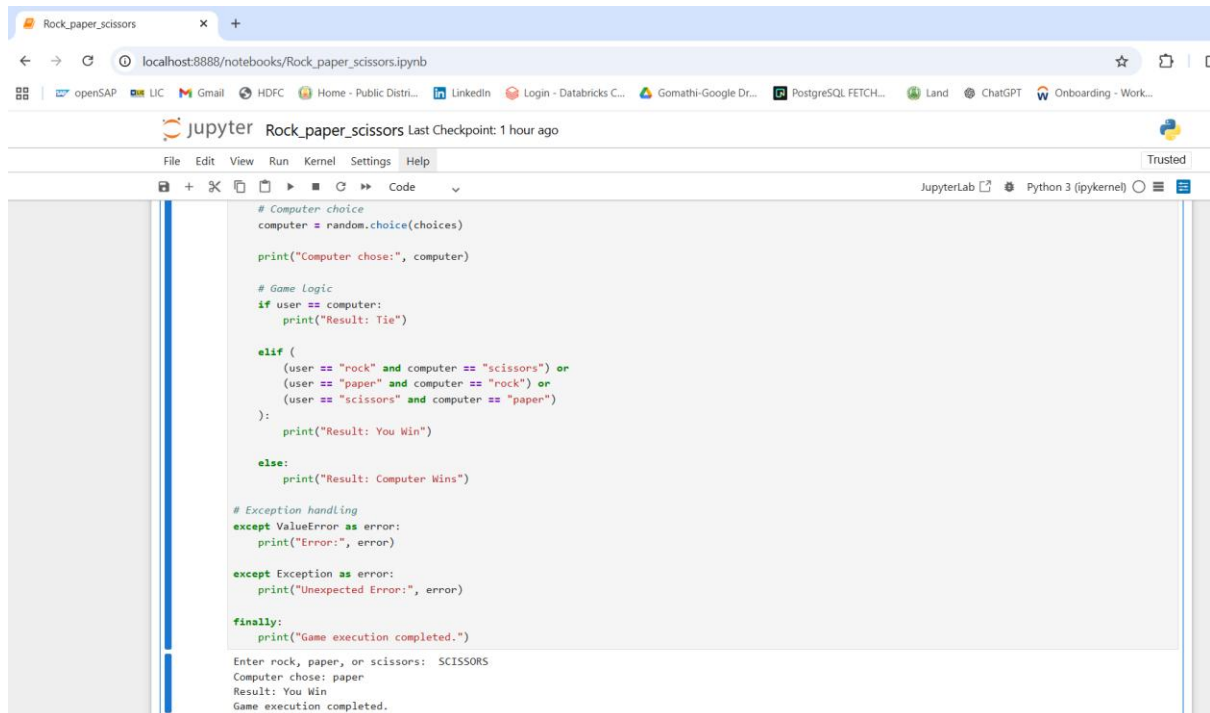
- Whether an error occurs or not

Purpose

- Cleanup operations
 - Final status display
 - Professional software practice
-

Output

#User Wins



The screenshot shows a JupyterLab notebook titled 'Rock_paper_scissors' running on a local host. The code defines a game logic where the user wins if they choose 'rock' and the computer chooses 'scissors', or if they choose 'paper' and the computer chooses 'rock', or if they choose 'scissors' and the computer chooses 'paper'. The output shows the user entering 'paper' and the game resulting in a win for the user.

```
# Computer choice
computer = random.choice(choices)

print("Computer chose:", computer)

# Game Logic
if user == computer:
    print("Result: Tie")

elif (
    (user == "rock" and computer == "scissors") or
    (user == "paper" and computer == "rock") or
    (user == "scissors" and computer == "paper")
):
    print("Result: You Win")

else:
    print("Result: Computer Wins")

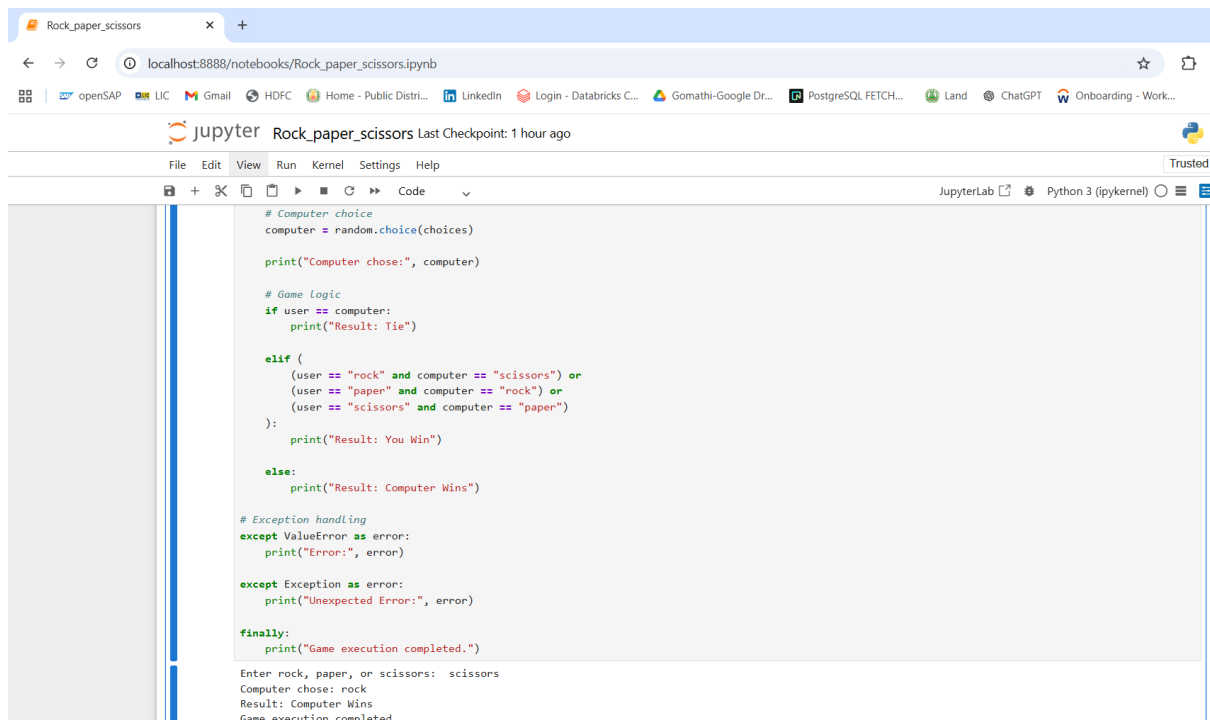
# Exception handling
except ValueError as error:
    print("Error:", error)

except Exception as error:
    print("Unexpected Error:", error)

finally:
    print("Game execution completed.")

Enter rock, paper, or scissors: SCISSORS
Computer chose: paper
Result: You Win
Game execution completed.
```

#Computer Wins



The screenshot shows the same JupyterLab notebook as above, but with the user entering 'scissors' and the computer choosing 'rock'. The output shows the game resulting in a win for the computer.

```
# Computer choice
computer = random.choice(choices)

print("Computer chose:", computer)

# Game Logic
if user == computer:
    print("Result: Tie")

elif (
    (user == "rock" and computer == "scissors") or
    (user == "paper" and computer == "rock") or
    (user == "scissors" and computer == "paper")
):
    print("Result: You Win")

else:
    print("Result: Computer Wins")

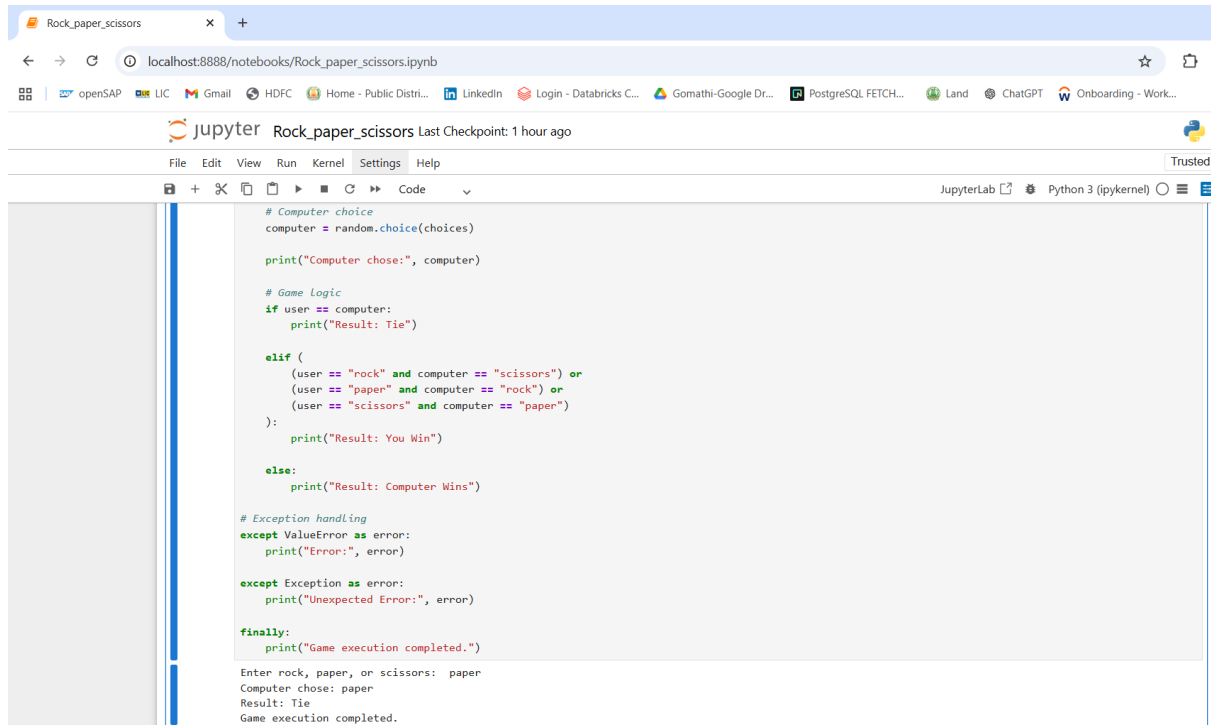
# Exception handling
except ValueError as error:
    print("Error:", error)

except Exception as error:
    print("Unexpected Error:", error)

finally:
    print("Game execution completed.")

Enter rock, paper, or scissors: scissors
Computer chose: rock
Result: Computer Wins
Game execution completed.
```

#Tie



```
# Computer choice
computer = random.choice(choices)

print("Computer chose:", computer)

# Game Logic
if user == computer:
    print("Result: Tie")

elif (
    (user == "rock" and computer == "scissors") or
    (user == "paper" and computer == "rock") or
    (user == "scissors" and computer == "paper")
):
    print("Result: You Win")

else:
    print("Result: Computer Wins")

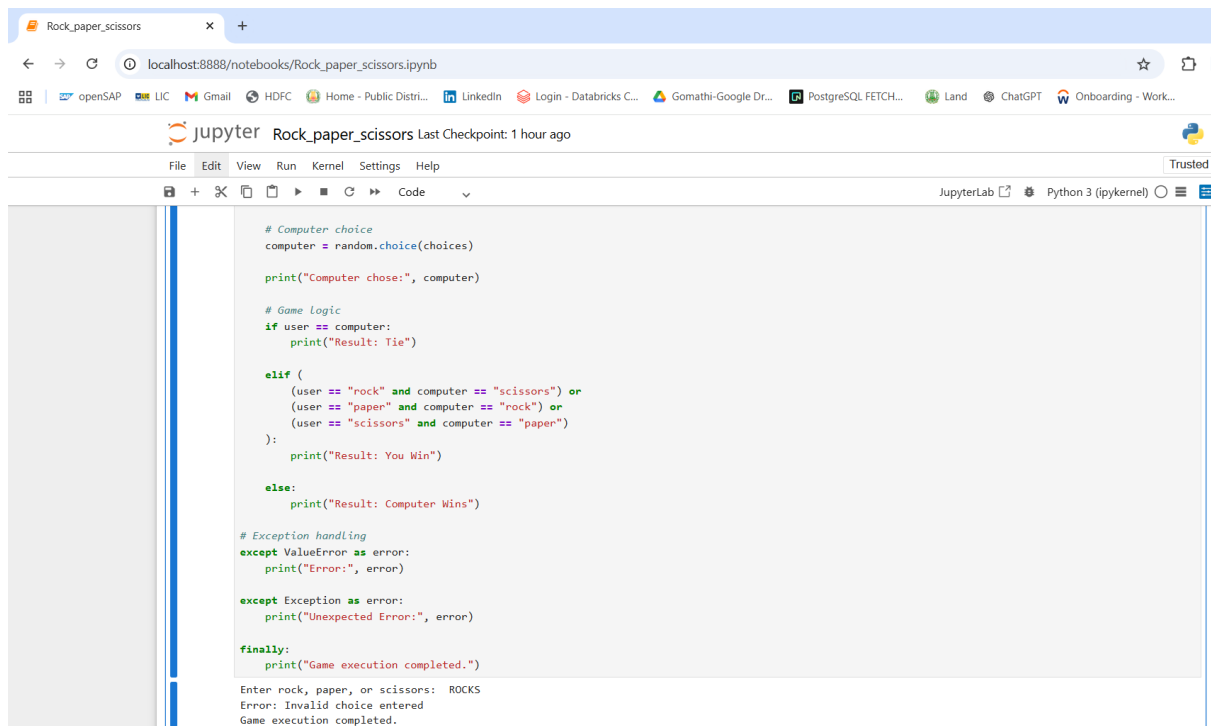
# Exception handling
except ValueError as error:
    print("Error:", error)

except Exception as error:
    print("Unexpected Error:", error)

finally:
    print("Game execution completed.")

Enter rock, paper, or scissors: paper
Computer chose: paper
Result: Tie
Game execution completed.
```

#INVALID INPUT(Instead of "ROCK", User entered "ROCKS")



```
# Computer choice
computer = random.choice(choices)

print("Computer chose:", computer)

# Game Logic
if user == computer:
    print("Result: Tie")

elif (
    (user == "rock" and computer == "scissors") or
    (user == "paper" and computer == "rock") or
    (user == "scissors" and computer == "paper")
):
    print("Result: You Win")

else:
    print("Result: Computer Wins")

# Exception handling
except ValueError as error:
    print("Error:", error)

except Exception as error:
    print("Unexpected Error:", error)

finally:
    print("Game execution completed.")

Enter rock, paper, or scissors: ROCKS
Error: Invalid choice entered
Game execution completed.
```

Conclusion

This source code demonstrates several important Python programming and software engineering concepts including:

- User interaction
- Randomization
- Decision making
- Input validation
- Exception handling
- Clean coding practices

The program is designed to be reliable, user-friendly, and easy to understand for beginners learning Python application development.

Limitations

- Text-based interface only
- No persistent score storage
- Single-round execution
- No multiplayer networking support

Future Enhancements

Proposed Improvements

- Graphical User Interface (GUI)
- Scoreboard system
- Multiplayer mode
- AI-powered prediction engine
- Database connectivity
- Web deployment

References

1. [Python Official Documentation](#)
Used for understanding Python syntax, modules, exception handling, and programming concepts.
2. [Python Random Module Documentation](#)
Referred for implementing random computer choices in the game.
3. [W3Schools Python Tutorial](#)
Used for learning Python basics, operators, lists, and conditional statements.
4. [GeeksforGeeks Python Programming](#)
Referred for programming concepts, exception handling, and coding examples.
5. [TutorialsPoint Python Tutorial](#)
Used for understanding Python modules and software engineering concepts.
6. Course materials and notes provided by Besant Technology.
7. Personal coding practice and implementation using Python in Jupyter Notebook.